Gamified Upper-Limb Stroke Rehabilitation System

Architecture, Project Plan and Implementation Guide

EMG + Computer Vision + Machine Learning + Rehabilitation Games + Remote Monitoring

Final Year Project documentation
October 2026

Contents

Part 1: Gamified Upper-Limb Stroke Rehabilitation System: System Architecture

1. Purpose
2. System Goals
3. High-Level Architecture
4. Layered View
5. Component Specifications
6. Data Architecture
7. Repository Structure
8. Technology Stack
9. Non-Functional Requirements
10. Key Risks and Design Responses
11. Evaluation Metrics
12. Future Extensions

Part 2: Project Plan: Gamified Upper-Limb Stroke Rehabilitation System

1. Project Summary
2. Objectives
3. Scope
4. Team Roles (adjust to team size)
5. Timeline (16 weeks)
6. Work Breakdown by Phase
7. Data Collection Plan
8. Evaluation Plan
9. Risk Register
10. Minimum Viable Product (MVP) vs. Stretch Goals
11. Ethics and Safety Checklist
12. Communication and Management
13. Report Outline
14. Definition of Done

Part 3: Implementation Guide: Gamified Upper-Limb Stroke Rehabilitation System

Step 0: Environment Setup
Step 1: ESP32 Firmware
Step 2: Data Acquisition Tools
Step 3: Preprocessing
Step 4: Feature Extraction
Step 5: Train and Evaluate ML Models
Step 6: Vision Module
Step 7: Fusion Engine
Step 8: Real-Time Pipeline and Command Bus
Step 9: Games
Step 10: Backend and Database
Step 11: Clinician Dashboard
Step 12: Calibration (recommended)
Step 13: Testing
Step 14: Latency Measurement
Step 15: Run Order Summary
Step 16: Troubleshooting
Step 17: Git Workflow
Final Checklist

Part 1: Gamified Upper-Limb Stroke Rehabilitation System: System Architecture

Version: 1.0 | **Scope:** FYP prototype (EMG + Computer Vision + AI/ML + 3 games + remote monitoring)

1. Purpose

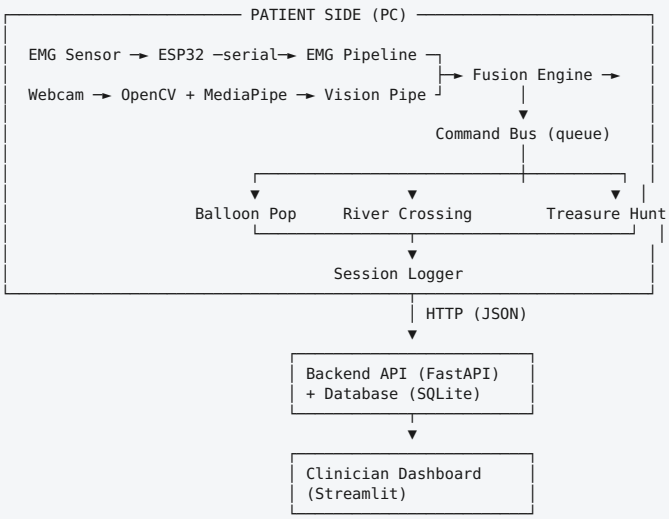
This document defines **what the system is made of and how the parts talk to each other**. For the schedule see `plan.md`; for build steps and code see `implement.md`.

2. System Goals

ID	Goal
G1	Detect 9 upper-limb movements from EMG and webcam input
G2	Use detected movements to control 3 rehabilitation games in real time (target latency < 300 ms)
G3	Give immediate visual feedback to the patient
G4	Log every session and show progress to a clinician remotely
G5	Keep every module independently testable (simulator and keyboard fallbacks)

Supported movements (class labels): `Hand_Open, Hand_Close, Wrist_Flexion, Wrist_Extension, Supination, Pronation, Rest, Left, Right`

3. High-Level Architecture



4. Layered View

Layer	Components	Responsibility
L1 Sensing	EMG sensor, electrodes, ESP32, webcam	Capture raw signals
L2 Signal Processing	Filters, windowing, landmark normalisation	Clean and segment data
L3 Feature + ML	Feature extractors, LDA/SVM/RF classifiers	Convert signals to movement labels
L4 Fusion	Fusion engine	Merge EMG and vision decisions into one command
L5 Application	Game engine and 3 games	Turn commands into rehabilitation exercises
L6 Data + Cloud	Session logger, API, database	Store and serve progress data
L7 Presentation	Patient UI (game), clinician dashboard	Feedback and monitoring

5. Component Specifications

5.1 Hardware

Part	Specification
Microcontroller	ESP32 DevKit, ADC1 pin (GPIO34/35/36/39). Do **not** use ADC2 pins when WiFi is on
EMG sensor	Analog EMG module (e.g. MyoWare 2.0), 1 or 2 channels
Electrodes	Ag/AgCl surface electrodes, reference electrode on bony area (elbow)
Power	Battery-powered sensor side recommended (reduces mains noise and improves patient safety)
Sampling rate	1000 Hz per channel, 12-bit ADC (0-4095)
Link to PC	USB serial at 921600 baud (WiFi/BLE as a later upgrade)

Electrode placement (suggested):

- Ch1: flexor carpi radialis / flexor digitorum (inner forearm)
- Ch2: extensor carpi radialis / extensor digitorum (outer forearm)
- Reference: elbow bony prominence

> **Design note:** A single EMG channel can separate *Rest vs. Active* reliably but not all 9 movements. This is why the architecture assigns movements to the sensor best suited to detect them (section 6). Use 2 channels if hardware allows.

5.2 EMG Pipeline

```
Raw ADC → remove DC offset → bandpass 20-450 Hz → notch 50 Hz → window (200 ms, 50% overlap)
        → features → scaler → classifier → (label, confidence)
```

Sri Lanka mains frequency is **50 Hz**, so use a 50 Hz notch filter.

Features per window per channel: RMS, MAV, VAR, STD, PEAK, WL, ZC, SSC.

5.3 Vision Pipeline

```
Frame → OpenCV (flip, RGB) → MediaPipe Hands (21 landmarks)
      → normalise (relative to wrist, scale by palm size)
      → rule-based or ML gesture classifier → (label, confidence)
```

Derived vision features: finger curl angles, fingertip-to-palm distances, wrist angle, palm-facing indicator, hand centre (x, y).

5.4 Fusion Engine

Each pipeline outputs `(label, confidence, timestamp)`. Fusion uses **role-based priority with confidence voting**:

1. **Rest gate (EMG):** if EMG says `Rest` with confidence ≥ threshold, output `Rest` (prevents accidental actions).
2. **Direction (vision):** `Left` / `Right` come from hand centre position in the frame.
3. **Grip (EMG + vision):** `Hand_Close` requires vision closed-hand **and** EMG activation. `Hand_Open` comes from vision with EMG low-to-moderate.
4. **Wrist and rotation:** `Wrist_Flexion/Extension` and `Supination/Pronation` use the highest-confidence source, with EMG channel balance and vision wrist angle as inputs.
5. **Smoothing:** majority vote over the last N=5 predictions, plus a minimum hold time (about 150 ms) to stop flickering.
6. **Output:** one `Command` object sent to the game.

5.5 Movement-to-Sensor Responsibility Matrix

Movement	Primary source	Secondary / confirmation	Notes
Rest	EMG	Vision (hand static)	Gate for all actions
Hand_Open	Vision	EMG (low activation)	Finger extension from landmarks
Hand_Close	Vision + EMG	n/a	Both must agree
Wrist_Flexion	EMG (flexor ch.)	Vision wrist angle	Hardest with 1 channel
Wrist_Extension	EMG (extensor ch.)	Vision wrist angle	Hardest with 1 channel
Supination	Vision (palm-facing)	EMG	Landmark z-order / palm normal
Pronation	Vision (palm-facing)	EMG	Landmark z-order / palm normal
Left	Vision	n/a	Hand centre x-position
Right	Vision	n/a	Hand centre x-position

5.6 Command Bus

A thread-safe `queue.Queue` carries commands from the real-time pipeline to the active game. Any source (ML pipeline, keyboard, simulator) can publish to it.

```
{
  "command": "Hand_Close",
  "confidence": 0.91,
  "source": "fusion",
  "timestamp_ms": 1759650000123
}
```

5.7 Games

All games inherit from `BaseGame` (input handling, score, timer, target-movement scheduler, logging).

Game	Controls	Rehab target
Balloon Pop	Left/Right move cursor, Hand_Open prepares, Hand_Close pops, Rest = no action	Grip/release, lateral control
River Crossing	Wrist_Flexion = down, Wrist_Extension = up, Left/Right = move, Rest = pause	Wrist control
Treasure Hunt	Left/Right move, Hand_Open selects, Hand_Close grabs, Supination/Pronation rotate	Coordination, forearm rotation

Each game defines a **target movement** per attempt, so every attempt can be logged as correct or incorrect.

5.8 Backend

- **Framework:** FastAPI
- **Database:** SQLite (PostgreSQL if scaled)
- **Auth (basic):** API key for the patient client, login for the clinician dashboard

Endpoints:

Method	Path	Purpose
POST	`/patients`	Create a patient
GET	`/patients`	List patients
POST	`/sessions`	Start a session
POST	`/sessions/{id}/attempts`	Upload attempt logs (batch)
PATCH	`/sessions/{id}`	Close session with summary
GET	`/patients/{id}/sessions`	Session history
GET	`/patients/{id}/progress`	Aggregated progress metrics

5.9 Database Schema

```
patients(patient_id PK, code, age, affected_side, stroke_date, notes)
sessions(session_id PK, patient_id FK, started_at, ended_at, game_name,
         total_score, duration_s, accuracy)
attempts(attempt_id PK, session_id FK, ts, target_movement, detected_movement,
         correct, reaction_time_ms, emg_rms, confidence)
```

Use coded IDs (P01, P02). **Never store names.**

5.10 Clinician Dashboard

Pages: **Patient list → Patient profile → Progress graphs (score, accuracy, reaction time per session) → Per-movement accuracy bar chart → Session table → Export CSV.**

6. Data Architecture

Dataset	File	Row =	Used for
Raw EMG	`emg_raw.csv`	one sample	Offline processing
EMG features	`emg_features.csv`	one window	Training EMG model
Hand landmarks	`hand_landmarks.csv`	one frame	Training/validating vision
Session log	`session_log.csv` / DB `attempts`	one attempt	Dashboard

Key columns: `subject_id, session_id, trial_id, timestamp_ms, emg_ch1, emg_ch2, x0..z20, label`.

Rule: split train/test **by subject or session**, never by random row, to avoid leakage.

7. Repository Structure

```
stroke-rehab-system/
├── README.md
├── requirements.txt
```

```
├── .gitignore
├── config/            settings.yaml, labels.yaml
├── firmware/emg_acquisition/emg_acquisition.ino
├── data/              raw/{emg,vision}, processed/, features/, logs/
├── src/
│   ├── acquisition/   emg_logger.py, landmark_recorder.py
│   ├── preprocessing/ filters.py, windowing.py
│   ├── features/      emg_features.py, vision_features.py
│   ├── models/        train_emg.py, train_vision.py, evaluate.py, saved/
│   ├── vision/        hand_tracker.py
│   ├── fusion/        fusion_engine.py
│   ├── realtime/      pipeline.py, simulator.py
│   ├── games/         game_engine.py, balloon_pop.py, river_crossing.py,
│   │                  treasure_hunt.py, assets/
│   └── utils/         logger.py, helpers.py
├── backend/           app.py, database.py, models.py, routes/
├── dashboard/         dashboard.py
├── notebooks/
├── tests/
├── docs/
└── results/           figures/, tables/
```

8. Technology Stack

Area	Tools
Firmware	Arduino IDE / PlatformIO
Signal processing	Python 3.10+, NumPy, SciPy, pandas
ML	scikit-learn, joblib
Vision	OpenCV, MediaPipe
Games	Pygame
Backend	FastAPI, Uvicorn, SQLite, SQLAlchemy
Dashboard	Streamlit, Plotly
Tooling	Git/GitHub, pytest, black

9. Non-Functional Requirements

Requirement	Target
End-to-end latency (movement → game response)	< 300 ms
EMG window	200 ms, step 100 ms
Vision frame rate	≥ 20 FPS
Offline-first	Game runs without internet; logs sync when online
Safety	Battery-powered/isolated EMG front end, no mains-connected electrodes
Privacy	Coded patient IDs, consent forms, no names or faces stored
Testability	Keyboard and simulator input for every game

10. Key Risks and Design Responses

Risk	Response
Single-channel EMG cannot separate all movements	Role-based sensor split, add 2nd channel, reduce classes if needed
Vision weak on wrist/rotation	Use palm-facing and wrist-angle features, EMG as confirmation
Cross-subject accuracy drops	Per-user calibration (30-60 s) at session start
Stroke patients: tremor, weak signal	Adjustable thresholds, difficulty levels, longer hold times
50 Hz noise, motion artefacts	Shielded leads, notch filter, battery power, good skin prep
Training on healthy subjects only	State clearly as a limitation; propose clinical validation as future work

11. Evaluation Metrics

- **Classification:** accuracy, precision, recall, F1, confusion matrix (subject-wise split)
- **System:** latency, FPS, command stability (flicker rate)
- **Usability:** task success rate, SUS questionnaire, patient/clinician feedback
- **Rehabilitation:** score and accuracy trend across sessions

12. Future Extensions

Multi-channel EMG armband, BLE/WiFi wireless link, cloud deployment, more games, adaptive difficulty driven by performance, clinical trial with stroke patients.

Part 2: Project Plan: Gamified Upper-Limb Stroke Rehabilitation System

Companion documents: `architecture.md` (design) and `implement.md` (build steps).

1. Project Summary

Item	Detail
Title	A Gamified Upper Limb Stroke Rehabilitation System with Remote Monitoring and Communication Features
Type	Final Year Project (hardware + AI/ML + software)
Duration	16 weeks (adjust to your semester)
Deliverables	Working prototype, datasets, trained models, 3 games, dashboard, report, presentation/viva

2. Objectives

1. Acquire EMG and hand-landmark data for 9 upper-limb movements.
2. Train and compare LDA, SVM and Random Forest classifiers.
3. Fuse EMG and vision outputs into one real-time command stream.
4. Build three rehabilitation games (Balloon Pop, River Crossing, Treasure Hunt).
5. Log sessions and show progress to a clinician through a dashboard.
6. Evaluate accuracy, latency and usability.

3. Scope

In scope: EMG acquisition (1-2 channels), webcam vision, ML classification, fusion, 3 PC games, local backend, dashboard, healthy-subject data collection.

Out of scope (future work): clinical trials on stroke patients, mobile app, multi-channel armband, cloud hosting, medical certification.

4. Team Roles (adjust to team size)

Role	Owner	Responsibilities
A: Hardware & Signals		ESP32, EMG circuit, firmware, filtering, data logger
B: ML		Features, training, evaluation, calibration
C: Vision & Fusion		MediaPipe tracker, gesture logic, fusion engine
D: Games & Monitoring		Game engine, 3 games, backend, dashboard
All		Testing, documentation, report sections

> For a solo project, follow the phase order and merge roles A+B, C+D.

5. Timeline (16 weeks)

Week	Phase	Tasks	Milestone
1	Planning	Finalise requirements, literature review, repo setup, ethics application	**M0:** Proposal and repo ready
2	Hardware	Wire ESP32 + EMG, write firmware, view live signal	**M1:** Clean EMG signal on PC
3	Acquisition	EMG logger, landmark recorder, data collection protocol, pilot recording	**M2:** Pilot dataset
4-5	Data collection	Record 5-10 subjects × 9 movements × 15 reps, 2 sessions	**M3:** Dataset complete
6	Preprocessing	Filters, windowing, feature extraction, dataset QA	**M4:** `emg_features.csv`
7-8	ML	Train LDA/SVM/RF, tune, subject-wise evaluation, pick best	**M5:** Trained EMG model + results
8-9	Vision	Hand tracker, derived features, gesture classifier	**M6:** Vision model/rules working
10	Fusion	Fusion engine, smoothing, real-time pipeline, simulator	**M7:** Live command stream
10-12	Games	Base engine, then Balloon Pop → River Crossing → Treasure Hunt	**M8:** 3 playable games
12-13	Backend & dashboard	API, DB, session logging, dashboard	**M9:** End-to-end monitoring

14	Integration	Full system test, latency check, bug fixing, calibration	**M10:** Working prototype
15	Evaluation	User testing, SUS survey, final results and figures	**M11:** Results ready
16	Documentation	Report, slides, demo video, viva preparation	**M12:** Final submission

Games (Weeks 10-12) can start earlier using the keyboard/simulator input, so they are **not blocked** by the ML work.

6. Work Breakdown by Phase

Phase 1: Planning

- [ ] Define the 9 classes and the movement-to-sensor mapping
- [ ] Literature review (EMG gesture recognition, stroke rehab games, MediaPipe)
- [ ] Submit ethics approval request and prepare consent form
- [ ] Create GitHub repo and folder structure

Phase 2: Hardware

- [ ] Build EMG front end and ESP32 wiring
- [ ] Firmware: 1000 Hz sampling, serial output
- [ ] Verify signal: rest is flat, contraction increases amplitude
- [ ] Document electrode placement with photos

Phase 3: Data Collection

- [ ] Write the data collection protocol
- [ ] Build the recording tool with on-screen prompts (movement, countdown, rest)
- [ ] Record all subjects, 2 sessions each
- [ ] Quality check: remove corrupted trials, check class balance

Phase 4: Processing and Features

- [ ] Bandpass and notch filtering
- [ ] 200 ms windows, 50% overlap
- [ ] Extract 8 features per channel
- [ ] Save `emg_features.csv`

Phase 5: Machine Learning

- [ ] Baseline: LDA, SVM (RBF), Random Forest
- [ ] Subject-wise cross-validation
- [ ] Hyperparameter tuning
- [ ] Confusion matrices, F1 per class
- [ ] Save best model with the scaler

Phase 6: Vision

- [ ] Real-time hand tracking with MediaPipe
- [ ] Rules or ML for Open/Close/Left/Right
- [ ] Palm-facing and wrist-angle features for rotation and wrist
- [ ] Record landmark dataset, validate accuracy

Phase 7: Fusion and Real-Time

- [ ] Implement fusion rules and smoothing
- [ ] Real-time loop with threads
- [ ] Simulator and keyboard fallback
- [ ] Measure latency

Phase 8: Games

- [ ] BaseGame (input, scoring, timer, target scheduler, logging)
- [ ] Balloon Pop
- [ ] River Crossing
- [ ] Treasure Hunt
- [ ] Difficulty levels and patient feedback visuals

Phase 9: Monitoring

- ☐ FastAPI backend and SQLite database
- ☐ Session upload from games
- ☐ Streamlit clinician dashboard

Phase 10: Testing, Evaluation and Documentation

- ☐ Unit tests, integration tests, end-to-end run
- ☐ User testing with 5+ volunteers, SUS questionnaire
- ☐ Final figures and tables
- ☐ Report, slides, demo video

7. Data Collection Plan

Item	Plan
Subjects	5-10 healthy volunteers (more is better)
Movements	9 classes
Repetitions	15 per movement per subject
Hold time	4 s active, 3 s rest between repetitions
Sessions	2 sessions on different days
Order	Randomised to avoid fatigue bias
Estimated size	10 × 9 × 15 ≈ 1,350 trials, thousands of windows
Per-subject time	About 30-40 min including setup

Checklist per subject: consent signed → skin cleaned → electrodes placed → signal check → record → backup file → note any issues.

8. Evaluation Plan

Area	Method	Target
EMG classification	Leave-one-subject-out CV	Report accuracy and F1 (aim > 85% on achievable classes)
Vision classification	Hold-out subjects	> 90% on Open/Close/Left/Right
Fusion	End-to-end test of 9 commands	Better than either source alone
Latency	Timestamp at sensor vs. game response	< 300 ms
Usability	SUS questionnaire, task completion	SUS > 68
Rehab tracking	Score and accuracy across sessions	Trend visible on dashboard

Treat the targets as goals. Report honest results even if they fall short, and explain why.

9. Risk Register

#	Risk	Likelihood	Impact	Mitigation
R1	Single EMG channel can't separate all classes	High	High	Add 2nd channel; assign classes by sensor; merge hard classes
R2	Noisy EMG (50 Hz, motion)	Medium	High	Notch filter, battery power, skin prep, shielding
R3	Ethics approval delay	Medium	High	Apply in Week 1; use team members first
R4	Poor cross-subject generalisation	High	Medium	Per-user calibration, more subjects
R5	Vision fails in poor lighting	Medium	Medium	Lighting guidelines, confidence threshold
R6	Latency too high	Medium	Medium	Smaller windows, threads, lighter model
R7	Hardware failure or lost parts	Low	High	Spare sensor and ESP32, backups
R8	Integration issues late	High	High	Simulator, weekly integration builds
R9	Time overrun	Medium	High	Prioritise (see MVP below), weekly review

10. Minimum Viable Product (MVP) vs. Stretch Goals

MVP (must finish):

- EMG Rest / Hand_Close detection plus vision Open/Close/Left/Right
- Balloon Pop fully working
- Session logging and a basic dashboard

- Honest evaluation and report

Target:

- All 9 movements with fusion
- All 3 games
- Calibration step, difficulty levels

Stretch:

- 2-channel EMG, wireless link, adaptive difficulty, patient trial

If time runs short, cut from the bottom up.

11. Ethics and Safety Checklist

- ☐ Ethics approval before recording anyone
- ☐ Written informed consent
- ☐ Coded IDs only, no names, no stored face video
- ☐ Battery-powered EMG device, never mains-connected while on a person
- ☐ Clean skin, single-use electrodes, stop if irritation occurs
- ☐ Data stored securely with limited access
- ☐ Clear statement that the prototype is **not a certified medical device**

12. Communication and Management

Practice	Detail
Weekly meeting	Review progress, blockers, next tasks
Supervisor update	Fortnightly with demo
Tracking	GitHub Projects or Trello board
Version control	`main` stable, feature branches, pull-request review
Logbook	Weekly entry: done, failed, decisions (feeds methodology chapter)

13. Report Outline

1. Introduction and problem statement
2. Literature review
3. System design (from `architecture.md`)
4. Methodology: data collection, processing, ML, fusion
5. Implementation: hardware, software, games, dashboard
6. Results and discussion
7. Limitations and future work
8. Conclusion
9. References and appendices (code, consent form, protocol)

14. Definition of Done

The project is complete when:

- The live system detects movements and controls all (MVP: at least one) games
- Session data appears on the dashboard
- Results are documented with figures and a confusion matrix
- Code is committed with a README that lets someone else run it
- Report, slides and demo video are submitted

Part 3: Implementation Guide: Gamified Upper-Limb Stroke Rehabilitation System

Follow the steps **in order**. Each step has a goal, code, and a check so you know it works. Code is a working starting point; adjust pins, ports and thresholds for your hardware. See `architecture.md` for design and `plan.md` for timing.

Step 0: Environment Setup

```
git init stroke-rehab-system && cd stroke-rehab-system
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate

mkdir -p config firmware/emg_acquisition data/{raw/emg,raw/vision,processed,features,logs} \
  src/{acquisition,preprocessing,features,models/saved,vision,fusion,realtime,games/assets,utils} \
  backend/routes dashboard notebooks tests docs results/{figures,tables}
touch src/__init__.py
```

requirements.txt

```
numpy
scipy
pandas
scikit-learn
joblib
pyserial
opencv-python
mediapipe
pygame
fastapi
uvicorn
sqlalchemy
streamlit
plotly
requests
pyyaml
pytest
matplotlib
```

```
pip install -r requirements.txt
```

.gitignore

```
venv/
__pycache__/
data/raw/
data/processed/
src/models/saved/
*.db
```

config/labels.yaml

```
labels: [Rest, Hand_Open, Hand_Close, Wrist_Flexion, Wrist_Extension,
         Supination, Pronation, Left, Right]
```

config/settings.yaml

```
serial_port: "COM3"        # Linux/Mac: /dev/ttyUSB0
baud_rate: 921600
fs: 1000                   # EMG sampling rate (Hz)
n_channels: 1
window_ms: 200
step_ms: 100
bandpass: [20, 450]
notch_hz: 50
smoothing_n: 5
min_confidence: 0.6
backend_url: "http://127.0.0.1:8000"
```

Step 1: ESP32 Firmware

Goal: stream EMG at 1000 Hz over serial.

`firmware/emg_acquisition/emg_acquisition.ino`

```
// ESP32 EMG acquisition: 1000 Hz, 1 channel (add a 2nd pin if needed)
const int EMG_PIN = 34;                // ADC1 pin
```

```
const unsigned long PERIOD_US = 1000;   // 1000 Hz
unsigned long nextT;

void setup() {
  Serial.begin(921600);
  analogReadResolution(12);             // 0..4095
  analogSetPinAttenuation(EMG_PIN, ADC_11db);
  nextT = micros();
}

void loop() {
  unsigned long now = micros();
  if ((long)(now - nextT) >= 0) {
    nextT += PERIOD_US;
    int v = analogRead(EMG_PIN);
    Serial.print(millis());
    Serial.print(',');
    Serial.println(v);
  }
}
```

Check: Arduino Serial Plotter shows a flat line at rest and larger spikes when you squeeze your fist.

Step 2: Data Acquisition Tools

2.1 EMG logger with on-screen prompts

`src/acquisition/emg_logger.py`

```
import serial, time, csv, random, yaml, os, sys

cfg = yaml.safe_load(open("config/settings.yaml"))
labels = yaml.safe_load(open("config/labels.yaml"))["labels"]

subject = sys.argv[1]            # e.g. S01
session = int(sys.argv[2])       # e.g. 1
REPS, HOLD, REST = 15, 4, 3      # reps per class, seconds

ser = serial.Serial(cfg["serial_port"], cfg["baud_rate"], timeout=1)
os.makedirs("data/raw/emg", exist_ok=True)
out = f"data/raw/emg/{subject}_session{session}.csv"

trials = [(l, r) for l in labels for r in range(1, REPS + 1)]
random.shuffle(trials)

def record(label, trial_id, seconds, writer):
    ser.reset_input_buffer()
    end = time.time() + seconds
    while time.time() < end:
        line = ser.readline().decode(errors="ignore").strip()
        parts = line.split(",")
        if len(parts) == 2 and parts[1].isdigit():
            writer.writerow([parts[0], subject, session, trial_id, parts[1], label])

with open(out, "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["timestamp_ms", "subject_id", "session_id", "trial_id", "emg_value", "label"])
    for i, (label, rep) in enumerate(trials, 1):
        print(f"\n[{i}/{len(trials)}] Get ready: {label}")
        time.sleep(2)
        print(f">>> DO: {label}")
        record(label, i, HOLD, w)
        print("... relax")
        record("Rest", i, REST, w)       # rest between reps adds Rest data
print("Saved", out)
```

Run: `python src/acquisition/emg_logger.py S01 1`

2.2 Landmark recorder

`src/acquisition/landmark_recorder.py`

```
import cv2, csv, sys, time
import mediapipe as mp

subject, session, label = sys.argv[1], int(sys.argv[2]), sys.argv[3]
hands = mp.solutions.hands.Hands(max_num_hands=1, min_detection_confidence=0.6)
cap = cv2.VideoCapture(0)
out = f"data/raw/vision/{subject}_session{session}_{label}.csv"

header = ["frame_id", "timestamp_ms", "subject_id", "session_id"] + \
         [f"{a}{i}" for i in range(21) for a in "xyz"] + ["label"]

with open(out, "w", newline="") as f:
    w = csv.writer(f); w.writerow(header)
    fid, t0 = 0, time.time()
    print("Press SPACE to start/stop recording, Q to quit")
    rec = False
    while True:
        ok, frame = cap.read()
        if not ok: break
```

```
        frame = cv2.flip(frame, 1)
        res = hands.process(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
        if rec and res.multi_hand_landmarks:
            lm = res.multi_hand_landmarks[0].landmark
            row = [fid, int((time.time() - t0) * 1000), subject, session]
            for p in lm: row += [p.x, p.y, p.z]
            w.writerow(row + [label]); fid += 1
        cv2.putText(frame, f"{label} REC={rec}", (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
        cv2.imshow("recorder", frame)
        k = cv2.waitKey(1) & 0xFF
        if k == ord(" "): rec = not rec
        if k == ord("q"): break
cap.release(); cv2.destroyAllWindows()
```

Run once per gesture: `python src/acquisition/landmark_recorder.py S01 1 Hand_Open`

Check: CSVs exist in `data/raw/`; class counts are roughly balanced.

Step 3: Preprocessing

`src/preprocessing/filters.py`

```
import numpy as np
from scipy.signal import butter, filtfilt, iirnotch

def bandpass(x, fs=1000, low=20, high=450, order=4):
    b, a = butter(order, [low/(fs/2), high/(fs/2)], btype="band")
    return filtfilt(b, a, x)

def notch(x, fs=1000, f0=50, q=30):
    b, a = iirnotch(f0/(fs/2), q)
    return filtfilt(b, a, x)

def clean(x, fs=1000):
    x = np.asarray(x, dtype=float)
    x = x - np.mean(x)            # remove DC offset
    return notch(bandpass(x, fs), fs)
```

`src/preprocessing/windowing.py`

```
import numpy as np

def make_windows(x, fs=1000, window_ms=200, step_ms=100):
    w, s = int(fs*window_ms/1000), int(fs*step_ms/1000)
    return [x[i:i+w] for i in range(0, len(x) - w + 1, s)]
```

Note: `filtfilt` is zero-phase and ideal for offline training. For real-time use, filter with `lfilter` and keep filter state (see Step 8).

Step 4: Feature Extraction

`src/features/emg_features.py`

```
import numpy as np

def extract(w):
    w = np.asarray(w, dtype=float)
    d = np.diff(w)
    thr = 0.01 * np.std(w) if np.std(w) > 0 else 0
    return {
        "RMS":  np.sqrt(np.mean(w**2)),
        "MAV":  np.mean(np.abs(w)),
        "VAR":  np.var(w),
        "STD":  np.std(w),
        "PEAK": np.max(np.abs(w)),
        "WL":   np.sum(np.abs(d)),
        "ZC":   int(np.sum((w[:-1]*w[1:] < 0) & (np.abs(w[:-1]-w[1:]) > thr))),
        "SSC":  int(np.sum((d[:-1]*d[1:] < 0) &
                           ((np.abs(d[:-1]) > thr) | (np.abs(d[1:]) > thr)))),
    }
```

Build the feature CSV: `src/features/build_dataset.py`

```
import glob, pandas as pd, yaml
from src.preprocessing.filters import clean
from src.preprocessing.windowing import make_windows
from src.features.emg_features import extract

cfg = yaml.safe_load(open("config/settings.yaml"))
rows = []
for path in glob.glob("data/raw/emg/*.csv"):
    df = pd.read_csv(path)
    for (subj, sess, trial), g in df.groupby(["subject_id", "session_id", "trial_id"]):
        # a trial may contain active + rest segments; process each label separately
        for label, seg in g.groupby("label", sort=False):
```

```
            if len(seg) < cfg["fs"] * cfg["window_ms"] / 1000: continue
            x = clean(seg["emg_value"].values, cfg["fs"])
            for k, w in enumerate(make_windows(x, cfg["fs"], cfg["window_ms"], cfg["step_ms"])):
                feats = extract(w)
                rows.append({"subject_id": subj, "session_id": sess, "trial_id": trial,
                             "window_id": k, **feats, "label": label})

out = pd.DataFrame(rows)
out.to_csv("data/features/emg_features.csv", index=False)
print(out.shape); print(out["label"].value_counts())
```

Run: `python -m src.features.build_dataset`

Check: `Rest` has clearly lower `RMS` than active classes. Class counts shown; fix imbalance if large.

Step 5: Train and Evaluate ML Models

`src/models/train_emg.py`

```
import pandas as pd, numpy as np, joblib
from sklearn.model_selection import LeaveOneGroupOut, cross_val_predict
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns   # pip install seaborn

df = pd.read_csv("data/features/emg_features.csv")
FEATS = ["RMS", "MAV", "VAR", "STD", "PEAK", "WL", "ZC", "SSC"]
X, y, groups = df[FEATS].values, df["label"].values, df["subject_id"].values

models = {
    "LDA": make_pipeline(StandardScaler(), LinearDiscriminantAnalysis()),
    "SVM": make_pipeline(StandardScaler(), SVC(kernel="rbf", C=10, probability=True)),
    "RF":  make_pipeline(StandardScaler(),
                         RandomForestClassifier(n_estimators=200, random_state=42)),
}

best_name, best_acc = None, 0
for name, m in models.items():
    pred = cross_val_predict(m, X, y, groups=groups, cv=LeaveOneGroupOut())  # subject-wise
    acc = accuracy_score(y, pred)
    print(f"\n=== {name}: accuracy {acc:.3f} ===")
    print(classification_report(y, pred))
    cm = confusion_matrix(y, pred, labels=sorted(set(y)))
    sns.heatmap(cm, annot=True, fmt="d", xticklabels=sorted(set(y)), yticklabels=sorted(set(y)))
    plt.title(name); plt.tight_layout()
    plt.savefig(f"results/figures/cm_{name}.png"); plt.clf()
    if acc > best_acc: best_name, best_acc = name, acc

print("Best:", best_name, best_acc)
final = models[best_name].fit(X, y)
joblib.dump(final, "src/models/saved/emg_model.joblib")
```

Notes

- If you have only one subject or session, use `GroupKFold` over `trial_id` instead (still no random row splits).
- If some classes (e.g. Flexion vs. Extension) confuse each other, report it honestly and consider the 2-channel upgrade or merging classes.

Check: confusion matrices saved in `results/figures/`; `emg_model.joblib` created.

Step 6: Vision Module

`src/vision/hand_tracker.py`

```
import cv2, numpy as np
import mediapipe as mp

mpH = mp.solutions.hands

def _angle(a, b, c):
    ba, bc = a - b, c - b
    cosv = np.dot(ba, bc) / (np.linalg.norm(ba) * np.linalg.norm(bc) + 1e-9)
    return np.degrees(np.arccos(np.clip(cosv, -1, 1)))

class HandTracker:
    def __init__(self):
        self.h = mpH.Hands(max_num_hands=1, min_detection_confidence=0.6,
                           min_tracking_confidence=0.6)

    def process(self, frame_bgr):
        frame = cv2.flip(frame_bgr, 1)
        res = self.h.process(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
        if not res.multi_hand_landmarks:
```

```
            return None
        lm = np.array([[p.x, p.y, p.z] for p in res.multi_hand_landmarks[0].landmark])
        return lm

    @staticmethod
    def features(lm):
        palm = np.linalg.norm(lm[0] - lm[9]) + 1e-9          # scale reference
        tips, pips, mcps = [8, 12, 16, 20], [6, 10, 14, 18], [5, 9, 13, 17]
        curl = [_angle(lm[m], lm[p], lm[t]) for t, p, m in zip(tips, pips, mcps)]
        dist = [np.linalg.norm(lm[t] - lm[0]) / palm for t in tips]
        cx, cy = lm[:, 0].mean(), lm[:, 1].mean()
        # palm normal z-component: sign hints supination vs pronation
        v1, v2 = lm[5] - lm[0], lm[17] - lm[0]
        palm_normal_z = np.cross(v1, v2)[2]
        wrist_angle = np.degrees(np.arctan2(lm[9][1] - lm[0][1], lm[9][0] - lm[0][0]))
        return {"curl": curl, "dist": dist, "cx": cx, "cy": cy,
                "palm_nz": palm_normal_z, "wrist_angle": wrist_angle}

def rule_classify(f):
    """Simple rule-based gesture classifier. Replace/augment with ML later."""
    mean_curl = np.mean(f["curl"])
    if f["cx"] < 0.35: return "Left", 0.8
    if f["cx"] > 0.65: return "Right", 0.8
    if mean_curl > 150: return "Hand_Open", 0.85      # straight fingers
    if mean_curl < 100: return "Hand_Close", 0.85     # curled fingers
    return "Rest", 0.5
```

Tuning: the thresholds (0.35/0.65, 150/100) must be calibrated on your own recordings. Print the features while you perform each gesture and pick thresholds from the data. For better accuracy, train a classifier (e.g. Random Forest) on `hand_landmarks.csv` features and save it as `vision_model.joblib`.

Check: a live window shows the correct label for open, close, left and right with a stable hand.

Step 7: Fusion Engine

`src/fusion/fusion_engine.py`

```
import time
from collections import deque, Counter

class FusionEngine:
    def __init__(self, smoothing_n=5, min_conf=0.6):
        self.buf = deque(maxlen=smoothing_n)
        self.min_conf = min_conf

    def fuse(self, emg, vis):
        """emg, vis: (label, confidence) or None"""
        e_lab, e_conf = emg if emg else ("Rest", 0.0)
        v_lab, v_conf = vis if vis else (None, 0.0)

        # 1. Rest gate
        if e_lab == "Rest" and e_conf >= self.min_conf and v_lab not in ("Left", "Right"):
            out = ("Rest", e_conf)
        # 2. Direction from vision
        elif v_lab in ("Left", "Right") and v_conf >= self.min_conf:
            out = (v_lab, v_conf)
        # 3. Grip: close needs both
        elif v_lab == "Hand_Close":
            out = ("Hand_Close", (v_conf + e_conf) / 2) if e_lab != "Rest" else ("Rest", e_conf)
        elif v_lab == "Hand_Open" and v_conf >= self.min_conf:
            out = ("Hand_Open", v_conf)
        # 4. Wrist / rotation: highest-confidence source
        elif e_lab in ("Wrist_Flexion", "Wrist_Extension", "Supination", "Pronation") \
                and e_conf >= self.min_conf:
            out = (e_lab, e_conf)
        elif v_lab in ("Supination", "Pronation", "Wrist_Flexion", "Wrist_Extension"):
            out = (v_lab, v_conf)
        else:
            out = (e_lab, e_conf)

        # 5. Smoothing: majority vote
        self.buf.append(out[0])
        label = Counter(self.buf).most_common(1)[0][0]
        return {"command": label, "confidence": out[1], "source": "fusion",
                "timestamp_ms": int(time.time() * 1000)}
```

Step 8: Real-Time Pipeline and Command Bus

`src/realtime/pipeline.py`

```
import threading, queue, time, yaml, joblib, serial, cv2, numpy as np
from collections import deque
from scipy.signal import butter, sosfilt, sosfilt_zi, iirnotch, tf2sos
from src.features.emg_features import extract
from src.vision.hand_tracker import HandTracker, rule_classify
from src.fusion.fusion_engine import FusionEngine

cfg = yaml.safe_load(open("config/settings.yaml"))
FEATS = ["RMS", "MAV", "VAR", "STD", "PEAK", "WL", "ZC", "SSC"]
```

```
command_bus = queue.Queue(maxsize=50)

latest = {"emg": None, "vis": None}
lock = threading.Lock()
stop = threading.Event()

def emg_thread():
    model = joblib.load("src/models/saved/emg_model.joblib")
    fs = cfg["fs"]; W = int(fs * cfg["window_ms"] / 1000); S = int(fs * cfg["step_ms"] / 1000)
    sos_bp = butter(4, [20/(fs/2), 450/(fs/2)], btype="band", output="sos")
    b, a = iirnotch(50/(fs/2), 30); sos_n = tf2sos(b, a)
    zi_bp = sosfilt_zi(sos_bp) * 0; zi_n = sosfilt_zi(sos_n) * 0
    ser = serial.Serial(cfg["serial_port"], cfg["baud_rate"], timeout=1)
    buf, count, dc = deque(maxlen=W), 0, 2048.0
    while not stop.is_set():
        parts = ser.readline().decode(errors="ignore").strip().split(",")
        if len(parts) != 2 or not parts[1].isdigit(): continue
        v = float(parts[1]); dc = 0.999 * dc + 0.001 * v          # running DC estimate
        y, zi_bp = sosfilt(sos_bp, [v - dc], zi=zi_bp)
        y, zi_n = sosfilt(sos_n, y, zi=zi_n)
        buf.append(y[0]); count += 1
        if len(buf) == W and count % S == 0:
            f = extract(np.array(buf))
            X = np.array([[f[k] for k in FEATS]])
            p = model.predict_proba(X)[0]
            with lock:
                latest["emg"] = (model.classes_[p.argmax()], float(p.max()))

def vision_thread():
    tr, cap = HandTracker(), cv2.VideoCapture(0)
    while not stop.is_set():
        ok, frame = cap.read()
        if not ok: continue
        lm = tr.process(frame)
        with lock:
            latest["vis"] = rule_classify(tr.features(lm)) if lm is not None else None
    cap.release()

def fusion_thread():
    fe = FusionEngine(cfg["smoothing_n"], cfg["min_confidence"])
    while not stop.is_set():
        with lock: emg, vis = latest["emg"], latest["vis"]
        cmd = fe.fuse(emg, vis)
        if not command_bus.full(): command_bus.put(cmd)
        time.sleep(0.05)                                          # ~20 Hz commands

def start():
    for t in (emg_thread, vision_thread, fusion_thread):
        threading.Thread(target=t, daemon=True).start()
```

Simulator (for development without hardware): `src/realtime/simulator.py`

```
import time, random, threading
from src.realtime.pipeline import command_bus

KEYS = ["Rest", "Hand_Open", "Hand_Close", "Wrist_Flexion", "Wrist_Extension",
        "Supination", "Pronation", "Left", "Right"]

def start_simulator(interval=1.0):
    def run():
        while True:
            command_bus.put({"command": random.choice(KEYS), "confidence": 1.0,
                             "source": "simulator", "timestamp_ms": int(time.time()*1000)})
            time.sleep(interval)
    threading.Thread(target=run, daemon=True).start()
```

Check: printing items from `command_bus` shows a stable, correct command while you move.

Step 9: Games

9.1 Base game

`src/games/game_engine.py`

```
import pygame, time, queue, random, requests, yaml

cfg = yaml.safe_load(open("config/settings.yaml"))

KEYMAP = {  # keyboard fallback so games work without ML
    pygame.K_LEFT: "Left", pygame.K_RIGHT: "Right", pygame.K_UP: "Wrist_Extension",
    pygame.K_DOWN: "Wrist_Flexion", pygame.K_o: "Hand_Open", pygame.K_c: "Hand_Close",
    pygame.K_s: "Supination", pygame.K_p: "Pronation", pygame.K_SPACE: "Rest",
}

class BaseGame:
    name = "Base"
    def __init__(self, bus, patient_id="P01", session_id=1, w=900, h=600):
        pygame.init()
        self.screen = pygame.display.set_mode((w, h)); self.W, self.H = w, h
        self.font = pygame.font.SysFont(None, 36)
        self.bus, self.patient, self.session = bus, patient_id, session_id
        self.score, self.attempts, self.running = 0, [], True
```

```
        self.cmd = "Rest"
        self.start_t = time.time()
        self.target, self.target_t = None, None

    def read_input(self):
        for e in pygame.event.get():
            if e.type == pygame.QUIT: self.running = False
            if e.type == pygame.KEYDOWN and e.key in KEYMAP: self.cmd = KEYMAP[e.key]
        try:
            while True: self.cmd = self.bus.get_nowait()["command"]
        except queue.Empty:
            pass

    def set_target(self, movement):
        self.target, self.target_t = movement, time.time()

    def log_attempt(self, detected):
        correct = int(detected == self.target)
        self.attempts.append({
            "ts": time.strftime("%Y-%m-%d %H:%M:%S"), "target_movement": self.target,
            "detected_movement": detected, "correct": correct,
            "reaction_time_ms": int((time.time() - self.target_t) * 1000), "emg_rms": 0.0})
        self.score += 10 if correct else 0
        return correct

    def hud(self):
        t = self.font.render(f"Score {self.score}   Target: {self.target}   Cmd: {self.cmd}",
                             True, (255, 255, 255))
        self.screen.blit(t, (10, 10))

    def save_session(self):
        data = {"patient_id": self.patient, "game_name": self.name,
                "total_score": self.score, "duration_s": int(time.time() - self.start_t),
                "attempts": self.attempts}
        try:
            requests.post(f"{cfg['backend_url']}/sessions", json=data, timeout=3)
        except Exception:
            import json; open("data/logs/pending_session.json", "a").write(json.dumps(data) + "\n")
```

9.2 Balloon Pop

`src/games/balloon_pop.py`

```
import pygame, random
from src.games.game_engine import BaseGame

class BalloonPop(BaseGame):
    name = "Balloon_Pop"

    def run(self):
        clock = pygame.time.Clock()
        cursor_x, balloons = self.W // 2, []
        self.set_target("Hand_Close")
        while self.running:
            self.read_input()
            if self.cmd == "Left":  cursor_x -= 8
            if self.cmd == "Right": cursor_x += 8
            cursor_x = max(0, min(self.W, cursor_x))
            if random.random() < 0.02:
                balloons.append([random.randint(40, self.W - 40), self.H])
            for b in balloons: b[1] -= 2
            self.screen.fill((20, 30, 60))
            for b in balloons: pygame.draw.circle(self.screen, (230, 60, 60), b, 30)
            pygame.draw.rect(self.screen, (255, 255, 0), (cursor_x - 15, 40, 30, 30))
            if self.cmd == "Hand_Close":                    # pop attempt
                hit = [b for b in balloons if abs(b[0] - cursor_x) < 40 and b[1] < 120]
                if hit:
                    balloons.remove(hit[0]); self.log_attempt("Hand_Close")
                    self.set_target("Hand_Close")
            self.hud(); pygame.display.flip(); clock.tick(30)
        self.save_session()
```

9.3 River Crossing and Treasure Hunt (same pattern)

- **River Crossing:** grid or lanes; `Wrist_Extension` moves up, `Wrist_Flexion` down, `Left/Right` sideways; obstacles spawn; reaching the far bank scores; `Rest` pauses. Set `target` to the move required to avoid the next obstacle.
- **Treasure Hunt:** move with `Left/Right`; `Hand_Open` selects a treasure; `Supination/Pronation` rotate a lock/key to the required angle; `Hand_Close` collects. Use a sequence target: `Hand_Open → rotate → Hand_Close`.

Both inherit `BaseGame`, call `set_target()` for each required movement, `log_attempt()` on each response, and `save_session()` at the end.

9.4 Launcher

`main.py`

```
import sys
from src.realtime import pipeline
from src.realtime.simulator import start_simulator
from src.games.balloon_pop import BalloonPop
```

```
mode = sys.argv[1] if len(sys.argv) > 1 else "keyboard"   # keyboard | sim | live
if mode == "live": pipeline.start()
elif mode == "sim": start_simulator()
BalloonPop(pipeline.command_bus).run()
```

Run: `python main.py keyboard` → `python main.py sim` → `python main.py live`

Check: the game is fully playable with the keyboard before you plug in any hardware.

Step 10: Backend and Database

`backend/app.py`

```
from fastapi import FastAPI
from pydantic import BaseModel
from typing import List
import sqlite3, datetime

app = FastAPI(title="Rehab Monitoring API")
DB = "rehab.db"

def db():
    c = sqlite3.connect(DB); c.row_factory = sqlite3.Row; return c

with db() as c:
    c.executescript("""
    CREATE TABLE IF NOT EXISTS sessions(
      session_id INTEGER PRIMARY KEY AUTOINCREMENT, patient_id TEXT, game_name TEXT,
      started_at TEXT, total_score INTEGER, duration_s INTEGER, accuracy REAL);
    CREATE TABLE IF NOT EXISTS attempts(
      attempt_id INTEGER PRIMARY KEY AUTOINCREMENT, session_id INTEGER, ts TEXT,
      target_movement TEXT, detected_movement TEXT, correct INTEGER,
      reaction_time_ms INTEGER, emg_rms REAL);
    """)

class Attempt(BaseModel):
    ts: str; target_movement: str; detected_movement: str
    correct: int; reaction_time_ms: int; emg_rms: float = 0.0

class SessionIn(BaseModel):
    patient_id: str; game_name: str; total_score: int; duration_s: int
    attempts: List[Attempt]

@app.post("/sessions")
def create_session(s: SessionIn):
    acc = sum(a.correct for a in s.attempts) / len(s.attempts) if s.attempts else 0
    with db() as c:
        cur = c.execute("INSERT INTO sessions(patient_id,game_name,started_at,total_score,"
                        "duration_s,accuracy) VALUES(?,?,?,?,?,?)",
                        (s.patient_id, s.game_name, datetime.datetime.now().isoformat(),
                         s.total_score, s.duration_s, acc))
        sid = cur.lastrowid
        c.executemany("INSERT INTO attempts(session_id,ts,target_movement,detected_movement,"
                      "correct,reaction_time_ms,emg_rms) VALUES(?,?,?,?,?,?,?)",
                      [(sid, a.ts, a.target_movement, a.detected_movement, a.correct,
                        a.reaction_time_ms, a.emg_rms) for a in s.attempts])
    return {"session_id": sid, "accuracy": acc}

@app.get("/patients/{pid}/sessions")
def sessions(pid: str):
    return [dict(r) for r in db().execute(
        "SELECT * FROM sessions WHERE patient_id=? ORDER BY started_at", (pid,))]

@app.get("/sessions/{sid}/attempts")
def attempts(sid: int):
    return [dict(r) for r in db().execute("SELECT * FROM attempts WHERE session_id=?", (sid,))]
```

Run: `uvicorn backend.app:app --reload` → API docs at `http://127.0.0.1:8000/docs`

Step 11: Clinician Dashboard

`dashboard/dashboard.py`

```
import streamlit as st, pandas as pd, requests, plotly.express as px

API = "http://127.0.0.1:8000"
st.set_page_config(page_title="Rehab Dashboard", layout="wide")
st.title("Clinician Dashboard: Stroke Rehabilitation")

pid = st.sidebar.text_input("Patient ID", "P01")
sessions = requests.get(f"{API}/patients/{pid}/sessions").json()

if not sessions:
    st.info("No sessions yet.")
else:
    df = pd.DataFrame(sessions)
    c1, c2, c3 = st.columns(3)
    c1.metric("Sessions", len(df))
    c2.metric("Latest score", int(df["total_score"].iloc[-1]))
```

```
    c3.metric("Latest accuracy", f"{df['accuracy'].iloc[-1]*100:.0f}%")

    st.plotly_chart(px.line(df, x="started_at", y="total_score", markers=True,
                            title="Score over time"), use_container_width=True)
    st.plotly_chart(px.line(df, x="started_at", y="accuracy", markers=True,
                            title="Accuracy over time"), use_container_width=True)

    sid = st.selectbox("Session", df["session_id"])
    att = pd.DataFrame(requests.get(f"{API}/sessions/{sid}/attempts").json())
    if not att.empty:
        per = att.groupby("target_movement")["correct"].mean().reset_index()
        st.plotly_chart(px.bar(per, x="target_movement", y="correct",
                               title="Accuracy per movement"), use_container_width=True)
        st.dataframe(att)
    st.download_button("Export sessions CSV", df.to_csv(index=False), "sessions.csv")
```

Run: `streamlit run dashboard/dashboard.py`

Step 12: Calibration (recommended)

Before each session, ask the patient to:

1. Relax for 10 s → record baseline `RMS_rest`.
2. Perform maximum comfortable contraction for 5 s → record `RMS_max`.
3. Normalise features: `RMS_norm = (RMS - RMS_rest) / (RMS_max - RMS_rest)`.

This reduces day-to-day and person-to-person variation and lets you set thresholds per patient (important for weak or inconsistent stroke muscle signals).

Step 13: Testing

`tests/test_features.py`

```
import numpy as np
from src.features.emg_features import extract

def test_rms_of_constant():
    f = extract(np.ones(200) * 5)
    assert abs(f["RMS"] - 5) < 1e-6

def test_rest_lower_than_active():
    rng = np.random.default_rng(0)
    assert extract(rng.normal(0, 1, 200))["RMS"] < extract(rng.normal(0, 10, 200))["RMS"]
```

`tests/test_fusion.py`

```
from src.fusion.fusion_engine import FusionEngine

def test_rest_gate():
    fe = FusionEngine(smoothing_n=1)
    assert fe.fuse(("Rest", 0.9), ("Hand_Close", 0.9))["command"] == "Rest"

def test_direction_from_vision():
    fe = FusionEngine(smoothing_n=1)
    assert fe.fuse(("Hand_Close", 0.8), ("Left", 0.9))["command"] == "Left"
```

Run: `pytest -q`

Integration test checklist

- ☐ Simulator → game → backend → dashboard works end-to-end
- ☐ Live EMG alone drives Rest/Hand_Close
- ☐ Vision alone drives Left/Right/Open/Close
- ☐ Fusion output is stable (no rapid flicker)
- ☐ Latency measured and recorded (< 300 ms target)
- ☐ Game keeps running if the backend is offline (pending log saved)

Step 14: Latency Measurement

Stamp each EMG window with `time.time()` when its last sample arrives, and log the time at which the game reacts to the resulting command. `latency = t_game − t_sample`. Report mean and 95th percentile over 100+ commands.

Step 15: Run Order Summary

```
# 1. Record data
```

```
python src/acquisition/emg_logger.py S01 1
python src/acquisition/landmark_recorder.py S01 1 Hand_Open     # repeat per gesture

# 2. Build features and train
python -m src.features.build_dataset
python src/models/train_emg.py

# 3. Start monitoring
uvicorn backend.app:app --reload
streamlit run dashboard/dashboard.py

# 4. Play
python main.py keyboard      # development
python main.py sim           # simulated commands
python main.py live          # EMG + camera + fusion
```

Step 16: Troubleshooting

Problem	Likely cause	Fix
Flat or saturated EMG	Poor electrode contact, wrong gain	Clean skin, check wiring, adjust sensor gain
Strong 50 Hz hum	Mains interference	Battery power, shorter leads, notch filter
ADC readings jump when WiFi on	ADC2 pin used	Use ADC1 pins (32-39)
Serial lines corrupted	Baud too high or buffer overrun	Lower baud, send in binary, check cable
Accuracy high in training, low live	Random split leakage, no calibration	Subject-wise split, add calibration
Commands flicker	No smoothing or low confidence	Majority vote, minimum hold time, raise threshold
MediaPipe misses hand	Lighting, background, distance	Good lighting, plain background, hand fully in frame
Latency too high	Heavy model, large window, slow loop	Smaller window, lighter model, separate threads

---

Step 17: Git Workflow

```
git checkout -b feature/emg-features
git add . && git commit -m "Add EMG feature extraction"
git push origin feature/emg-features      # open a pull request into main
```

Commit small and often. Never commit `data/raw/` or trained models.

Final Checklist

- ☐ Hardware streams clean EMG
- ☐ Dataset recorded, balanced, quality-checked
- ☐ Models trained with subject-wise evaluation, figures saved
- ☐ Vision tracker stable
- ☐ Fusion engine tested
- ☐ 3 games playable (keyboard, simulator and live)
- ☐ Backend and dashboard working end to end
- ☐ Tests pass, latency measured
- ☐ README, report, slides, demo video complete